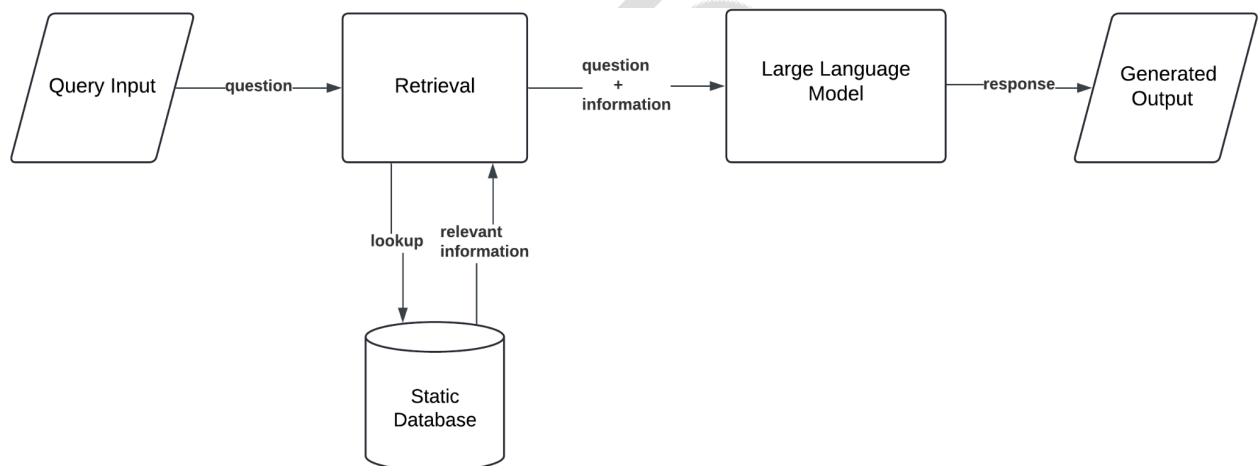


SUMMARY: BUILDING BLOCKS OF RAG

Objective:

1. Understand why chunking documents is essential for effective retrieval in RAG.
2. Distinguish between fixed-size, hierarchical, and semantic chunking strategies.
3. Identify common retrieval pitfalls like redundancy and missing context.
4. Recognize the value of overlap, semantic breaks, and metadata in chunking.
5. Apply retrieval-side strategies such as similarity metrics, thresholding, MMR, and re-ranking.
6. Appreciate how advanced frameworks like LangChain and LlamaIndex simplify best practices in chunking and retrieval.

Revisiting RAG:



1. The importance of Architecture and Pipeline

Now that we understand what RAG does, let's go one level deeper and see **how** it's actually structured in production systems. This leads us to the **Retriever-Generator Pipeline Architecture**, which makes RAG reliable, interpretable, and scalable — especially in real-world applications

We'll try to understand Retriever and Generators separately.

So far, we've been using **prompt templates** in Python to interact with LLMs. This worked well — but only when **all the required information was already in the prompt or known by the LLM**.

But in real-world use cases, this isn't enough.

Instead of just giving the LLM a query, we now need to:

- a. **Search for the most relevant information** (retrieve)
- b. **Feed that information + the query to the LLM** (generate)

Instead of just this:

```
prompt = f"Answer this question: {user_query}"
response = llm(prompt)
```

We now have:

```
docs = retriever.retrieve(user_query) # Step 1: Get the best context
context = format_docs(docs) # Step 2: Format context for LLM
prompt = f"Using the information below, answer the
question:\n{context}\n\nQ: {user_query}"
response = llm(prompt) # Step 3: Generate grounded
response
```

This is **Retriever-Generator** work alongside.

```
User Query
↓
Retriever → Top-k Documents
↓
Prompt + Docs → Generator (LLM)
```

↓
Generated Answer (grounded in real context)

1.1 It's Not Just About Retrieving Documents and Passing Them to an LLM

Many beginners believe RAG simply means:

1. Find relevant documents using vector similarity.
2. Paste them into a prompt.
3. Ask the LLM to answer.

But real-world systems are **far more complex** — and fragile — if we don't design them carefully.

1.2 Problems with Retriever

Even if you're retrieving documents using vector embeddings and passing the best ones as context:

Problem 1: Irrelevant or misleading retrieval

Vector similarity might return **passages that look similar but aren't truly relevant**.

For example: You search for *"loan renewal rules in Tier 2 cities"* and get documents about *"loan approval in rural areas"*. Similar wording, but different context

This happens when your retriever lacks **semantic depth** — it can't *truly understand* the meaning of the query.

Problem 2: Top-k chunks may not capture the complete picture

Even if you get the “top-5” most similar documents:

- They may be **redundant**
- They may miss **critical nuance**
- They may be **spread across multiple documents**, which the LLM struggles to stitch together

So just retrieving the “closest” documents isn't enough. You need smarter retrieval strategies and chunking.

Problem 3: Query and Document Embeddings Not Aligned

If your query and documents are not encoded into the **same kind of semantic space**, your retriever might not understand what you're looking for. It's like asking a question in Hindi, and your retriever looking for answers in English based on literal translation.

Without alignment, **vector search fails silently** — and gives wrong answers with high confidence

1.3 Problems with the Generator (LLM) Side

Once documents are retrieved, the generator is expected to:

- Read the context
- Understand the query
- Produce a clear, relevant, and grounded answer

But even here, problems arise:

Problem 1: LLM gets confused by noisy or excessive context

If the retrieved content is:

- Too long
- Poorly structured
- Repetitive

The LLM may generate vague or incorrect responses.

LLMs do not reason across dozens of passages reliably, especially if some contradict each other

Problem 2: LLM still hallucinates

Even with context, the LLM might:

- Make up facts
- Misinterpret numbers
- Confuse similar-sounding entities

Why? Because it doesn't “check” documents like a search engine — it *generates* based on patterns, not verification.

1.4 Why You Can't Treat RAG as a Black Box in Production

If students treat RAG as a “black box,” they face following problems:

1. You Can't Debug What You Don't Understand
2. You Can't Scale Blindly
3. You Can't Explain or Justify Model Outputs
4. You Can't Optimize Retrieval or Generation Independently

Understanding the RAG pipeline isn't optional. It's the difference between a working prototype and a production-ready system.

1.5 What Does Understanding Architecture Teach?

It teaches you that building AI systems isn't just about prompts or LLMs — it's about designing a modular, explainable pipeline where every component plays a role.

You learn to:

- Think about retriever quality, not just LLM quality
- Align query and document embeddings
- Format context to help, not hinder the generator

So we broadly classify this section into 4 parts:

1. Revisiting RAGs
2. Understanding the Granularities of Retriever
3. Understanding the Granularities of Generator
4. Crafting Prompts with RAGs

2. RAG (revisited)

2.1 Modular Components Working Together

[Transition] - Until now, you've seen that retrieving documents and passing them as context to an LLM is not enough. You've understood the why. Now, let's understand the how — how real, production-grade Retrieval-Augmented Generation (RAG) systems are built under the hood.

At its core, the RAG architecture consists of two clearly separated but tightly connected engines:

Let's try to understand with an example:

Imagine you're a developer building an AI assistant for a lending app used by call center agents. The app should help agents quickly answer questions about:

- Loan eligibility
- Offer limits
- Document requirements
- Internal rule changes

All the information exists — but it’s buried inside multiple sources: PDFs, policy change logs, product manuals, and internal communication threads. Manually crafting a prompt won’t scale or stay accurate for long.

You need a system that can:

- Understand the query
- Retrieve the most relevant content from internal documents
- Generate a reliable, fluent response

This is exactly what the **Retriever-Generator pipeline** enables.

The Retriever: Focused Information Access

The retriever’s job is to **search large document collections efficiently** and return only the most relevant information.

Technically, this is done using **dense vector representations** of both the query and the documents. These vectors capture **semantic meaning**, not just keyword overlap.

In practice:

- You process your corpus (e.g., loan rules, policy terms) into document chunks.
- Each chunk is embedded into a dense vector using a **pretrained encoder**.
- When a query comes in, it is also embedded using the same model.
- A **similarity search** (usually cosine similarity) returns the **top-k most relevant chunks** from the vector database.

For example, for the query:

“Can a customer apply for another loan within 30 days of their last disbursal?”

The retriever should ideally return:

- Policy documents about waiting periods
- Internal memos on cool-down windows
- Notes on system validations for repeat loan applications

Without a reliable retriever, the generator receives low-quality context, and your entire pipeline suffers.

The Generator: Fluent, Grounded Output

The generator’s job is to **take the retrieved documents and the original query** and produce a well-formed response.

It typically uses a **sequence-to-sequence model** such as BART, T5, or GPT to:

- Read and combine the retrieved information
- Understand the user's intent
- Generate a clear, grounded answer

For example:

“As per current policy, customers must wait 45 days between disbursements. Exceptions apply only to salaried borrowers with pre-approved limits.”

This model doesn't “know” the answer on its own — it builds the answer using the retrieved inputs, reasoning across them.

Why This Modularity Matters

The separation between retriever and generator is not just clean design — it's necessary for real-world scalability, transparency, and maintainability.

Modularity

- You can fine-tune the retriever on domain-specific corpora without touching the generator.
- You can upgrade the generator model without changing retrieval logic.

Scalability

- The retriever can scale to millions of documents using vector databases like FAISS or Weaviate.
- The generator only works with the top-k retrieved chunks, keeping inference fast and efficient.

Transparency

- You can log which chunks were retrieved and why.
- You can debug bad generations by seeing if retrieval failed or generation misinterpreted.

In a production environment, this visibility is essential for reliability and compliance — especially when handling financial or legal documents.

Building AI Systems, Not Just Prompts

When you adopt the Retriever-Generator pipeline, you move from prompt experimentation to **system design**. You now think in terms of:

- Retrieval quality
- Embedding alignment
- Chunking and formatting

- Generation logic and fallback mechanisms

This pipeline forces clarity of responsibility:

- Retrieval solves **"What to say?"**
- Generation solves **"How to say it?"**

And both together ensure that what you say is **relevant, accurate, and aligned with your source of truth.**

2.2 Query and Document Embedding: Same Vector Space Representation

So far, you've understood that the retriever finds relevant chunks using embeddings. But for that to actually work, there's one critical requirement:

The query and the documents must be embedded into the same vector space.

This isn't a nice-to-have. It's the foundation. If this fails, your retriever will quietly return irrelevant, misleading, or generic content — and your entire pipeline will break downstream without obvious errors.

Why It Matters

Imagine you're building an AI assistant for a loan servicing platform. A user asks:

"Can a co-applicant remove themselves after disbursal?"

If your documents were embedded using a general-purpose model and the query is embedded using a fine-tuned legal language model, you've introduced a mismatch.

The query vector is now living in a different coordinate system than the documents. Even though they're both "vectors," they're not semantically aligned.

You might retrieve:

- A paragraph about KYC updates
- A policy on missed EMIs
- A section on loan top-ups

But not the actual clause explaining co-applicant rights.

The system returns chunks that sound close, but aren't meaningful. And your LLM will try to reason over incomplete or incorrect input — grounding on the wrong foundation.

What Does "Same Vector Space" Mean?

It means:

- The same encoder model (architecture + weights) was used to embed both queries and documents.
- Both are transformed using the same rules, vocabulary, and semantics.
- The similarity metric (like cosine similarity) now has **semantic value**.

When that's true, the retriever can measure which documents *mean* the same thing as the query — not just share words with it.

What Happens When You Ignore This?

The failure is quiet but damaging:

- Retrieval still runs.
- Similarity scores are returned.
- But results are poor, and you're debugging the LLM instead of fixing retrieval.

It's like plotting latitude and longitude from different map projections — technically valid, but directionally useless.

What Developers Should Always Do

- Use **the same embedding model** for both document indexing and incoming query encoding.
- Avoid mixing models (e.g., OpenAI for documents, Hugging Face for queries).
- Ensure embeddings are domain-relevant — especially in finance, insurance, healthcare, or legal applications.

Retriever Architecture

[Transition] So to retrieve the best possible documents or text from available database, we need to:

1. Convert input query to embeddings
2. Convert Document to embeddings
3. Match and retrieve best docs.

User query → query embedding → vector similarity → retrieve docs → pass to generator → generate response

But we talked of some problems that can occur if it's that simple:

1. Retrieving similar looking but irrelevant docs.
2. Retrieving top-k chunks that are redundant or incomplete

3. Query and doc embeddings not aligned
4. Generator confused by long/noisy content
5. Still hallucinates/guesses.

This sets up **why retrieval is not just "get top 5 documents"**.

1. Document side strategies

In RAG systems, everyone talks about the LLM or the prompt. But before the LLM ever sees anything, your system has already made a high-stakes decision:

What parts of your documents are worth showing?

That decision happens at the **document side** — where you chunk, embed, and prepare your knowledge for retrieval.

If you mess this up, it doesn't matter how smart your retriever is or how advanced your LLM is. You're feeding it the wrong input — and the whole pipeline will produce confident nonsense.

Two Parts of Document-Side Strategy

We'll break this into:

1. **Text Chunking Strategies** — how to break the document for embedding
2. **Indexing Considerations (light)** — how the chunks are stored and retrieved later[To be discussed in detail with vector Data Bases]

Let's focus primarily on chunking — that's where most real problems begin.

1.1 Text Chunking Strategies

Solving the "What exactly should we embed?" problem

Large documents can't be embedded/turned into vectors or searched all at once.

You split them into chunks — but how you split them changes everything.

That's **text chunking**.

Why It's Not Just About Size. Students often imagine chunking like:

"Split every 300 tokens and done."

But what if 300 tokens cuts a policy rule in half? What if two critical details are split across separate chunks?

Suddenly, your query matches nothing — even though your content is there.

Imagine a policy document from a digital lending platform that includes the following text:

Chunk A:

“Loan renewal is available for Tier 1 and Tier 2 cities. Customers must have a repayment score above 700 and no defaults in the last 12 months. Maximum renewal amount is...”

Chunk B:

“...₹1,00,000 for salaried customers and ₹50,000 for self-employed customers. However, customers in rural areas are only eligible for fresh loans, not renewals.”

Let's say you split this document **naively every 300 tokens**, and it cuts off right between Chunk A and B.

Now imagine a user query:

“Can self-employed customers in Tier 2 cities apply for loan renewal?”

What Happens?

If only Chunk B is retrieved (because of keywords like “self-employed” and “loan renewal”), the system might only see:

“₹50,000 for self-employed customers. However, customers in rural areas are only eligible for fresh loans...”

Result:

The system may wrongly assume that the customer is *not eligible for renewal* — because the chunk with the Tier 2 condition was cut off or missed.

If only Chunk A is retrieved, the system might wrongly return:

“Yes, Tier 2 customers can renew loans up to the limit.”

But without the self-employed detail.

Bad chunking is one of the most common silent failures in RAG systems.

You won't see an error. You'll just get wrong or vague answers.

Let's walk through chunking strategies -

Strategy	What It Does	When It Works Well	Pitfalls
Fixed-size chunking	Breaks text into chunks of fixed token count (e.g. 300) with overlap (10–15%)	Fast, easy to implement	May split sentences or related ideas
Hierarchical chunking	Breaks by document structure (e.g., section → paragraph → sentence)	Great for manuals, reports	Needs parsing logic
Semantic chunking	Uses embedding distance to split only when topic changes	Preserves meaning	Slower and requires experimentation

1.2 Demonstrating with a code

```
pip install nltk sentence-transformers scikit-learn
```

```
import re
from nltk.tokenize import sent_tokenize
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Sample input text
sample_text = """
Section 1: Loan Eligibility
To qualify for a personal loan, the customer must be between 21 and 60
years old, have a minimum salary of ₹25,000, and a credit score above 700.

Section 2: Disbursal and Timelines
Loan disbursal typically occurs within 48 hours of approval. Delays can
occur due to incomplete documentation.

Section 3: EMI Defaults
If the customer misses 2 or more EMIs, penalties apply. Further defaults
```

may lead to legal action or freezing of accounts.

Section 4: Foreclosure

Customers can foreclose their loan after 6 EMIs have been paid. A foreclosure fee of 2% is applicable.

"""

1. Fixed-Size Chunking

```
def fixed_size_chunking(text, chunk_size=300, overlap=50):
```

"""

Split text into fixed-size chunks with optional overlap (measured in tokens).

"""

tokens = text.split()

chunks = []

start = 0

while start < len(tokens):

end = start + chunk_size

chunk = ' '.join(tokens[start:end])

chunks.append(chunk)

start += chunk_size - overlap

return chunks

2. Hierarchical Chunking

```
def hierarchical_chunking(text):
```

"""

Split document using structured section headers (e.g., "Section 1:", "Section 2:").

"""

sections = re.split(r'\n(?:=Section \d+:)', text.strip())

return [sec.strip() for sec in sections if sec.strip()]

3. Semantic Chunking

```
def semantic_chunking(text, threshold=0.75):
```

"""

Group nearby sentences based on semantic similarity (using cosine

```
similarity).
"""
    model = SentenceTransformer('all-MiniLM-L6-v2')
    sentences = sent_tokenize(text)
    embeddings = model.encode(sentences)

    chunks = []
    current_chunk = [sentences[0]]
    for i in range(1, len(sentences)):
        sim = cosine_similarity([embeddings[i]], [embeddings[i-1]])[0][0]
        if sim >= threshold:
            current_chunk.append(sentences[i])
        else:
            chunks.append(' '.join(current_chunk))
            current_chunk = [sentences[i]]
    if current_chunk:
        chunks.append(' '.join(current_chunk))
    return chunks

# -----
# Display Utility
# -----
def display_chunks(chunks, title):
    print(f"\n{title} -- Total Chunks: {len(chunks)}")
    for i, chunk in enumerate(chunks):
        print(f"\n--- Chunk {i+1} ---\n{chunk}")

# -----
# Run All Chunkers
# -----
fixed_chunks = fixed_size_chunking(sample_text, chunk_size=40, overlap=10)
hierarchical_chunks = hierarchical_chunking(sample_text)
semantic_chunks = semantic_chunking(sample_text, threshold=0.75)

display_chunks(fixed_chunks, "Fixed-Size Chunking")
display_chunks(hierarchical_chunks, "Hierarchical Chunking")
display_chunks(semantic_chunks, "Semantic Chunking")
```

1.3 What Problem does Chunking Solve?

Problem 1: Irrelevant or Misleading Retrieval

- When documents are chunked semantically or hierarchically, each chunk stays on-topic.
- This helps the retriever return meaningful, focused snippets — not bloated general ones.

Problem 2: Top-k Retrieval Misses Critical Info

- Overlapping fixed chunks catch details that land between boundaries.
- Semantic chunking ensures each chunk is a coherent thought, not mid-sentence noise.
- Hierarchical chunking respects the document's structure — sections stay together.

Problem 3: LLM Confusion from Bad Context

- Smaller chunks = more chances to mix and match the best content.
- Well-labeled sections let the LLM “see” topic boundaries.
- Cleaner context means fewer hallucinations and better grounding.

Without chunking:

You're handing the retriever and LLM a messy room and asking them to find a needle in the haystack.

With chunking:

You're organizing the room into neat labeled drawers. Retrieval becomes precise. Generation becomes coherent.

1.4 What Comes After Basic Chunking?

Now that you understand why chunking is important and how to perform it using fixed-size, semantic, and hierarchical methods, the next step is to optimize it for production-ready applications.

In real-world scenarios, you'll encounter challenges such as:

- Large volumes of documents with varied formats
- Documents that are updated frequently (e.g., policies, support manuals)
- System constraints like memory, latency, and token limits
- Use-case-specific needs, where one chunking strategy doesn't fit all

This leads us to advanced practices, such as:

- Adaptive chunking based on the LLM's input window
- Intelligent filtering of low-value chunks
- Cross-chunk stitching or scoring based on context or metadata
- Semantic enrichment of chunks using entity or topic detection

These are complex to build from scratch. Fortunately, two major tools are designed to handle this effectively.

1.5 Introducing LangChain and LlamaIndex

a. LangChain

LangChain is a high-level framework built to help developers create end-to-end LLM applications. It simplifies:

- Text chunking with control over token size and overlap
- Attaching metadata to chunks for better filtering or tracking
- Seamless integration of retrievers and LLMs in a pipeline
- Managing prompt templates and output parsing

LangChain provides various ready-to-use chunking utilities such as **RecursiveCharacterTextSplitter**, **TokenTextSplitter**, and more.

You can choose whether to split by characters, tokens, or custom rules, and immediately embed or store them for search.

2. LlamaIndex (formerly GPT Index)

LlamaIndex focuses on connecting structured data—like PDFs, markdown files, and databases—to language models.

Its key features include:

- Intelligent parsing of large documents into nodes (chunks)
- Support for hierarchical document structures (titles, sub-sections, etc.)
- Auto-updating indices when documents change
- Semantic scoring and chunk retrieval via customizable query engines

LlamaIndex is especially useful when dealing with documents that have natural hierarchy or when you need greater control over document indexing and retrieval logic.

Because you've understood the **why** behind chunking, these tools offer the **how** for production environments. They allow you to:

- Move from isolated chunking experiments to fully integrated pipelines
- Rapidly test which chunking strategies give the best grounding
- Build scalable RAG systems with configurable retrieval and generation logic

In upcoming sections, we'll explore how these tools operationalize chunking, context management, and retrieval in ways that support robust, flexible RAG architectures.

2. Retrieval side strategies

So far, you've tackled **"How to convert documents into embeddings?"**

That's the **document side**: chunking strategies, preserving meaning, and ensuring useful context is available.

But that's only half the battle.

Now we move to **"How do we pull the best pieces out?"**

This is the **retrieval side**—where things like similarity scores, diversity, and ranking logic determine **what actually gets shown to the LLM**.

Here are some strategies to pull out best scores:

1. Similarity Metrics: What Does "Relevant" Actually Mean?

Most developers think, "My retriever will just find the closest chunk." But what does "closest" mean in high-dimensional space?

Imagine two vectors—one for your query, one for each chunk. Are you comparing **distance**, **direction**, or something else entirely.

- **Cosine similarity**: Think of vectors as arrows; cosine measures how parallel they are. Closer to 1 means more "pointed in the same direction"—semantically similar.
- **Dot product**: Cares about both the length and direction. Sometimes exaggerates the match if chunks or queries are long.

- **Euclidean distance:** Like a ruler—smaller is better, but doesn’t always fit the “meaning” of language.

You ask:

“What are the penalties for EMI defaults?”

- Chunk A: “EMI defaults attract a penalty of 2% per month.”
- Chunk B: “Penalties for late documentation submission...”
- Chunk C: “EMI holidays are available for salaried employees.”

Cosine similarity ensures **Chunk A** ranks highest—not just by keyword, but by context.

If your similarity metric is off, the retriever returns “almost relevant” chunks—setting up your LLM to hallucinate or miss the real answer.

You feel like you “almost got it”—but the answer’s always just out of reach.

2.2. Similarity Score Thresholding: Filtering Out the Fuzz

You don’t want “maybe related” context.

Just because two pieces of text share words doesn’t mean they’re relevant.

Imagine you retrieve five chunks for every question—no matter the similarity score. That’s like inviting five random people to answer a technical interview. How many will actually help?

- Set a threshold: only include chunks above, say, 0.55 similarity.
- Anything lower is treated as “not useful.”

Your LLM is given these chunks for the query,

“How soon can I renew my loan?”

- Chunk with score 0.83: “Loans can be renewed after 6 months.”
- Chunk with score 0.44: “Our branches in Delhi offer evening hours.”

If you don’t filter, the LLM may mention irrelevant facts, confuse geographies, or give generic advice.

Research and production RAG systems routinely use thresholds—often determined via **empirical tuning** on real user queries.

Ask students:

“What happens if you set the threshold too high? Too low?”

When to Use Similarity Score Thresholding

- When you want to include only highly relevant context and filter out weak or unrelated chunks.
- When your data is broad, noisy, or covers many topics.
- When you notice vague, off-topic, or hallucinated answers from your LLM.
- In user-facing, compliance, or high-stakes applications where precision is critical.

When Not to Use (or Use Cautiously)

- When your data is already tightly scoped and retrieval is precise.
- When every retrieved chunk is always relevant (rare).
- When a high threshold might accidentally exclude important context—always tune and test for your use case.

2.3 Maximum Marginal Relevance (MMR): Mixing Relevance and Diversity

When a user submits a query to your RAG system, your retriever pulls out the “top-k” most similar chunks from your document database.

But here’s the catch:

The default top-5 are often almost the same — slight rephrasings, from the same section, or all about one narrow aspect.

The result? Your LLM keeps seeing the same fact, over and over.

Why is this a problem?

- The LLM may overfocus and answer only that one aspect.
- Important, but less “loud” information gets ignored.
- If the query is broad (“Tell me about eligibility, penalties, and renewal”), vanilla retrieval won’t provide coverage.

What does MMR do?

MMR (Maximum Marginal Relevance) changes the game by ensuring that every chunk you add is not only **relevant**, but also **adds something new**.

It works like this:

- First, select the most relevant chunk.
- Next, for each remaining candidate chunk, score it not just by how relevant it is to the query, but also how *different* it is from what’s already selected.

- Continue until you've picked k chunks, always balancing *relevance* and *diversity*.

Suppose the user asks:

“What are the age, salary, and credit score requirements for a personal loan?”

Your chunk pool might include:

- **Chunk 1:** “Applicants must be at least 21 years old.”
- **Chunk 2:** “Minimum monthly salary is ₹25,000.”
- **Chunk 3:** “Applicants must have a credit score above 700.”
- **Chunk 4:** “Applicants must be at least 21 years old and earn ₹25,000.”
- **Chunk 5:** “Minimum credit score of 700 is required.”

Vanilla top-k:

You might get Chunks 1, 4, and 5—all repeating the age/score requirements, missing salary.

MMR:

You'd get Chunk 1 (age), Chunk 2 (salary), Chunk 3 (credit score)—one chunk per important aspect, less repetition, better coverage.

How Does MMR Work in Practice?

- MMR uses a formula to balance similarity to the query and dissimilarity to already-picked chunks.
- A tuning parameter (λ) lets you dial the trade-off:
 - Closer to 1 = prioritize query relevance
 - Closer to 0 = prioritize novelty/diversity

Advantages

- Provides a much broader answer set to the LLM
- Reduces redundancy—less “echo chamber” effect
- Helps the LLM ground answers in more facets of the question, especially when the query is multi-topic

Disadvantages

- Slightly more compute — more pairwise comparisons between chunks
- If λ is set wrong, you might sacrifice relevance for diversity, including off-topic chunks
- Requires thoughtful tuning and testing on your dataset

When to Use MMR

- Whenever your queries are open-ended, cover multiple facts, or need comprehensive answers (banking, legal, policy, HR, insurance)
- When your top-k chunks are often redundant, and you notice the LLM is repetitive

When Not to Use

- Very focused queries, or when you trust that your chunking already segments by topics well
- Small document sets with little risk of repetition

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# -----
# 1. Sample Data
# -----

chunks = [
    "Applicants must be at least 21 years old.",
    "Minimum monthly salary is ₹25,000.",
    "Applicants must have a credit score above 700.",
    "Applicants must be at least 21 years old and earn ₹25,000.",
    "Minimum credit score of 700 is required.",
    "Personal loans are not available to self-employed applicants.",
    "EMI defaults attract a penalty of 2% per month.",
]

query = "What are the age, salary, and credit score requirements for a
personal loan?"

# -----
# 2. Embedding
# -----

model = SentenceTransformer('all-MiniLM-L6-v2')
chunk_embeddings = model.encode(chunks)
query_embedding = model.encode([query])[0]

# -----
# 3. Vanilla Top-k Retrieval
```

```
# -----  
  
def vanilla_top_k(chunk_embeddings, query_embedding, k=3):  
    similarities = cosine_similarity([query_embedding],  
chunk_embeddings)[0]  
    top_k_idx = similarities.argsort()[-k:][::-1]  
    return [chunks[i] for i in top_k_idx], similarities[top_k_idx]  
  
vanilla_results, vanilla_scores = vanilla_top_k(chunk_embeddings,  
query_embedding, k=3)  
print("\nVanilla Top-3 Results:")  
for text, score in zip(vanilla_results, vanilla_scores):  
    print(f" - {text} [score: {score:.2f}]")  
  
# -----  
# 4. MMR-based Retrieval  
# -----  
  
def mmr(chunk_embeddings, query_embedding, k=3, lambda_param=0.7):  
    """  
    Returns k chunks using Maximum Marginal Relevance (MMR)  
    """  
    # Calculate cosine similarities  
    similarities = cosine_similarity(chunk_embeddings,  
[query_embedding]).flatten()  
    selected = []  
    selected_idx = []  
    candidate_idx = list(range(len(chunks)))  
  
    for _ in range(k):  
        if len(selected_idx) == 0:  
            idx = np.argmax(similarities)  
            selected_idx.append(idx)  
            selected.append(chunks[idx])  
            candidate_idx.remove(idx)  
            continue  
        # For each candidate, calculate max similarity with already  
        selected  
        diversity = np.array([  
            max(cosine_similarity([chunk_embeddings[i]],  
[chunk_embeddings[j]])[0][0] for j in selected_idx)  
            for i in candidate_idx
```

```
    ])
    mmr_score = lambda_param * similarities[candidate_idx] - (1 -
lambda_param) * diversity
    idx = candidate_idx[np.argmax(mmr_score)]
    selected_idx.append(idx)
    selected.append(chunks[idx])
    candidate_idx.remove(idx)
    return selected

mmr_results = mmr(chunk_embeddings, query_embedding, k=3, lambda_param=0.7)
print("\nMMR Top-3 Results:")
for text in mmr_results:
    print(f" - {text}")
```

2.4 Re-ranking and Metadata Filtering: Making Retrieval Context-Aware

Let's say you've solved redundancy with MMR, but not all chunks are equally useful right now. Some chunks are outdated, others are internal notes, some are FAQ snippets, and some are from your latest policy update.

How Does Re-ranking Work?

After retrieving a set of candidate chunks (whether via vanilla, thresholded, or MMR-based retrieval), you can:

- **Prioritize** chunks with a “recent” timestamp
- **Exclude** those labeled “INTERNAL ONLY” or “DRAFT”
- **Promote** chunks marked as “FAQ” if the query looks like a FAQ
- Use additional rules: prefer documents tagged as “official”, prefer those with positive user feedback, etc.

Example: Policy Change and User Query

User asks: “What is the current minimum salary for a personal loan?”

You retrieve:

- **Chunk A:** “As of 2018, the minimum salary was ₹20,000.”
- **Chunk B:** “As of April 2024, the minimum salary is ₹25,000 for salaried professionals.”
- **Chunk C:** “Draft: Considering raising minimum to ₹28,000 next year.”

Without re-ranking, the retriever may serve the oldest chunk first—outdated information.

With re-ranking, you ensure **Chunk B**—the most current and applicable—shows up first.

Advantages

- Keeps answers current and compliant
- Avoids embarrassing “draft” or internal info leaking into responses
- Makes your system adaptable to business rules, regulations, and even user preferences

Disadvantages

- Requires careful metadata tagging and regular document updates
- Risk of over-filtering (excluding a relevant chunk by mistake)
- Adds complexity to your retrieval code and pipeline

When to Use Re-ranking

- When your data has meaningful metadata (date, type, confidentiality, status)
- When freshness, compliance, or user context matter

When Not to Use

- All data is equally relevant, current, and public (rare in business settings)

```
# Each chunk now has some metadata, e.g., year and type
chunks_metadata = [
    {"text": "Applicants must be at least 21 years old.", "year": 2018,
    "type": "policy"},
    {"text": "Minimum monthly salary is ₹25,000.", "year": 2024, "type":
    "policy"},
    {"text": "Applicants must have a credit score above 700.", "year":
    2024, "type": "policy"},
    {"text": "Applicants must be at least 21 years old and earn ₹25,000.",
    "year": 2019, "type": "policy"},
    {"text": "Minimum credit score of 700 is required.", "year": 2018,
    "type": "faq"},
    {"text": "Personal loans are not available to self-employed
    applicants.", "year": 2023, "type": "faq"},
```



```
    {"text": "EMI defaults attract a penalty of 2% per month.", "year":  
2022, "type": "policy"},  
]
```

```
chunks = [item["text"] for item in chunks_metadata]
```

```
from sentence_transformers import SentenceTransformer  
from sklearn.metrics.pairwise import cosine_similarity  
import numpy as np
```

```
query = "What are the age, salary, and credit score requirements for a  
personal loan?"
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')  
chunk_embeddings = model.encode(chunks)  
query_embedding = model.encode([query])[0]
```

```
def vanilla_top_k(chunk_embeddings, query_embedding, k=3):  
    similarities = cosine_similarity([query_embedding],  
chunk_embeddings)[0]  
    top_k_idx = similarities.argsort()[-k:][::-1]  
    return [chunks[i] for i in top_k_idx], top_k_idx,  
similarities[top_k_idx]
```

```
vanilla_results, top_k_idx, vanilla_scores =  
vanilla_top_k(chunk_embeddings, query_embedding, k=3)  
print("\nVanilla Top-3 Results (before re-ranking):")  
for text, score in zip(vanilla_results, vanilla_scores):  
    print(f" - {text} [score: {score:.2f}]")
```

```
# Let's re-rank the retrieved top-k results by 'year' (descending, newest  
first)
```

```
def rerank_by_year(top_k_idx, chunks_metadata):  
    # Extract tuples (index, year)  
    idx_years = [(idx, chunks_metadata[idx]["year"]) for idx in top_k_idx]  
    # Sort by year descending  
    reranked = sorted(idx_years, key=lambda x: x[1], reverse=True)  
    reranked_texts = [chunks_metadata[idx]["text"] for idx, _ in reranked]  
    return reranked_texts
```

```
reranked_results = rerank_by_year(top_k_idx, chunks_metadata)

print("\nRe-ranked Top-3 Results (by recency):")
for text in reranked_results:
    print(f" - {text}")
```

Generator Architecture: Understanding how Generator works

Transition - So far, we've seen that once we retrieve the most relevant documents, we typically **add them as a string** before the user query and send the whole prompt to the model.

This is how systems like **ChatGPT or Gemini** typically work. These are examples of **decoder-only generator architectures**.

They:

- Read the context and the query **in one go**, or trim it or summarize it depending on strategy.
- Treat everything as one long input string,
- And **start generating** the answer right after that — continuing the sequence token by token.

This approach is powerful, simple, and works well for many applications — especially if the context is short and well-structured.

Before we move forward let's first try to understand what are encoders and decoders.

1. Encoders and Decoders: Generator Perspective in RAG

1.1 What is an Encoder?

An **encoder** is a neural network (or part of a model) whose job is to take input data (like text, images, or audio) and convert it into a compressed, meaningful numerical representation—called an **embedding** or **latent vector**.

Key ideas:

- The encoder “reads” or “understands” the input.
- It turns complex, variable-length input (like a sentence or document) into a fixed-size vector that summarizes its meaning.
- This vector is a dense, information-rich summary—capturing the important features, relationships, and semantics.

Examples in practice:

- In language models, an encoder might read a paragraph and encode its meaning as a vector.
- In machine translation, the encoder reads a sentence in French and turns it into a vector representing the meaning, ready for translation.

1.2 What is a Decoder?

A **decoder** is a neural network (or part of a model) whose job is to take a numerical representation (like the encoder’s output vector) and generate a desired output sequence—often new text, images, or reconstructed data.

Key ideas:

- The decoder “speaks” or “writes” based on the encoded information.
- In language, the decoder might generate a sentence, word by word, using the vector produced by the encoder.
- It can turn a compressed idea back into something understandable for humans.

Examples in practice:

- In translation, the decoder takes the vector from the French sentence and writes out the English translation.
- In image captioning, the decoder turns image features (from the encoder) into a descriptive sentence.

1.3 Where Are Encoders and Decoders Used?

In a RAG pipeline, there are two separate “sides” that use neural models:

1. **Retriever Side:**

Encoders turn documents and queries into dense vectors (embeddings) to find relevant chunks (e.g., using BERT, MiniLM, etc.).

2. **Generator Side:**

Encoders and/or *Decoders* process the input context and generate the natural-language answer.

It’s critical not to confuse these two:

- The retriever’s encoder makes *searchable vector representations*.

- The generator's encoder/decoder (if present) does *deep language understanding and answer generation*.

1. The Encoder - Decoder Architecture

Let's try to understand different ways in which Generator can generate outputs.

1.1 Decoder only Models & Encoder only models

What we have learnt so far are decoder only architectures. They retrieve necessary documents and give it as context to LLMs like GPT.

Encoder only Architecture:

Encoder-only architectures, like BERT, are designed for deep bidirectional context understanding, making them highly effective for tasks such as classification, sentiment analysis, and named entity recognition. They excel at understanding text rather than generating it, as they lack a decoder to produce new sequences. Since Large Language Models (LLMs) require both understanding and generation capabilities, encoder-only models are not used for LLMs. Instead, LLMs rely on decoder-only or encoder-decoder architectures for text generation tasks.

Decoder-Only Architecture (like GPT, ChatGPT)

- There is **no separate encoder**. (Although the retriever still uses vector embeddings)
The model sees the **entire input sequence** (retrieved documents + query) as just one string of tokens.
- These tokens are **embedded** into vectors right before generation.
- The model then starts **generating the output immediately**, using those embedded vectors — but **not through a separate memory**. It just continues the sequence. In short, when generating output, the model just keeps going—**autoregressively**—from where the input ends i.e every new token is predicted based on all tokens that came before it, including both context and user query.

Example: What Does This Look Like in Practice?

Suppose you have this context **and** query:

Context:

Applicants must be at least **21** years old.

Minimum salary **is** ₹**25,000**/month.

User Query:

What **is** the minimum salary **for** a personal loan?

Prompt for Decoder-Only Model (GPT-style after info is retrieved using vectors):

```
Applicants must be at least 21 years old.  
Minimum salary is ₹25,000/month.
```

```
What is the minimum salary for a personal loan?
```

Model sees this as a single sequence: [Context tokens][Query tokens]

- When the model generates a response, it does not “load” a separate memory of the context.
- Instead, it uses the entire previous sequence as its basis for predicting the next token, just as if it were writing a story or continuing a transcript.

From the Transformer/Autoregressive Perspective

- **Autoregressive:**
The model always predicts the next token based on **all tokens seen so far**.
There’s no checkpoint or “fixed state” passed from encoder to decoder.
It’s like reading everything so far and deciding what comes next—again and again.
- **No Boundary:**
The model doesn’t “know” where context ends and the answer begins, except via the pattern in the text (like a separator, “Answer:” prompt, or new line).
You can prompt it:

```
[Context]  
[User Query]  
Answer:
```

and the model will start generating after “Answer:” simply by continuing the sequence.

Think of a group chat:

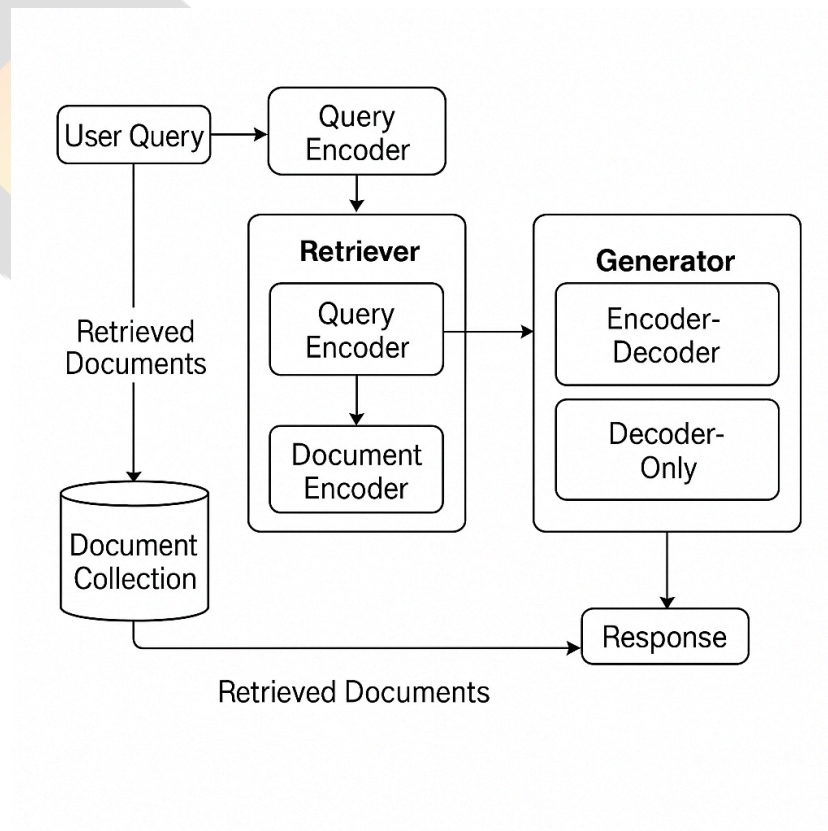
- In encoder-decoder, one person summarizes all the previous messages (context), hands you their notes, and you reply based on the notes.
- In decoder-only, you just read the entire chat history and then type your response, one word at a time, without ever pausing to summarize. You always look back at the whole conversation so far, every time you type.

You can think of it as:

“Read and write in one pass — generate the answer as an immediate continuation of the input.”

Please note - Vectors are used to bring the relevant information and then all the information is given to Generator, the generator processes it like a string - this is **Decoder only generator model**.

1.2 Encoder-decoder generator architecture



a. What It Is

An encoder-decoder generator is a language model architecture used for generation tasks where:

- The input is first processed (encoded) into a deep semantic representation
- The output is then generated (decoded) from that representation

This architecture is common in models like T5, BART, and Pegasus, especially for tasks like:

- Summarization
- Translation
- Multi-document question answering
- Structured reasoning

Think of it as a two-stage generator:

- First: Understand the input deeply (using vector representations)

- Then: Generate the output based on that understanding

b. How It Works — Step by Step

Let's say the user asks:

“Can a customer apply again within 30 days of loan disbursement?”

And you've retrieved 3 relevant document chunks. Now you pass these to the **encoder-decoder generator model**.

Step 1: Encoding the Input

You concatenate your retrieved chunks and user query like:

```
Document 1: Customers must wait 45 days between loan disbursements.  
Document 2: Exceptions apply to salaried customers.  
Question: Can a customer apply again within 30 days?
```

- The **encoder** processes this entire input and converts each token into a **vector representation** that captures its contextual meaning.
- It builds a full understanding of:
 - What the policy says
 - What the question is asking
 - How the retrieved info connects

Important - You first use vector embeddings to find relevant documents and then those documents are again encoded. No output is generated yet. The encoder is like a smart reader — it's building a complete internal map of the input.

Step 2: Decoding the Output

- The **decoder** now kicks in.
- It receives the encoded vector representations from the encoder.
- It starts generating the answer **token by token**, and at each step.

For example, it may generate:

"No, customers must wait 45 days before applying again, except if they are salaried and pre-approved."

This output is not guessed. It's built with full awareness of all the retrieved context, thanks to the encoder-decoder structure.

c. When and Why Use Encoder-Decoder Generators

Scenario	Why Encoder-Decoder Helps (Detailed)
Multi-document summarization	The encoder can read, process, and compress information from multiple sources, capturing key points and discarding noise before the decoder ever begins to write. This separation allows the model to digest a lot of context, reason over it, and avoid being overwhelmed by input length. The decoder then creates a concise summary grounded in the encoder's comprehensive "memory."
Structured output needed	Because the encoder forms a unified representation of the input, the decoder can generate complex, well-organized outputs—like bullet lists, tables, or forms. It's easier to control structure since the model separates "understanding" (encoder) from "formatting" (decoder), which helps avoid blending sections or losing structure in the output.
Complex Q&A requiring synthesis	The encoder brings together information scattered across the input, allowing the decoder to synthesize connections between different facts, resolve ambiguities, and produce well-reasoned answers. This is valuable for technical support, law, or science, where answers are rarely in a single chunk but must be inferred across sources.
Translation or rewriting	Encoder-decoder models read the entire source (sentence, paragraph, document) to fully grasp its meaning, context, and tone. The decoder can then generate a high-fidelity translation or rewrite, preserving nuance, idioms, and intent—rather than simply continuing a sequence token by token.

d. Comparison to Decoder-Only Models

Feature	Encoder-Decoder Generator	Decoder-Only Generator	Why/How (Detailed Explanation)

Input understanding	Deep, pre-generation encoding	Inline, during generation	Encoder-decoder models process and fully “digest” all input using the encoder before generation starts, building a global understanding of the context. Decoder-only models take in the entire prompt (context + question) as a single sequence and process it token by token during generation—understanding unfolds as it writes.
Model stages	Separate encoder and decoder	Single model block	There’s a clear separation: one network encodes the input, another generates the output, allowing focused optimization for each. In decoder-only models, everything (reading and writing) is handled by one model “block,” making the architecture and flow simpler and reducing coordination steps.
Control over structure	More (via encoder guidance)	Less (relies on prompt design)	The encoder can be guided to produce outputs in specific structures (like tables, summaries, or bullet lists) because it globally understands the input first. Decoder-only models depend on how well the prompt “hints” at the desired format—if the prompt is vague, output may lose structure or mix topics.
Simplicity of use	Needs formatting + setup	Just prompt → generate	Encoder-decoder models require you to separately prepare input for the encoder and manage decoder setup (often more code or config). Decoder-only models need only a single prompt with everything inline—retrieved chunks, question, instructions—making them fast to deploy, easy to update, and friendlier for experimentation or prototyping.
Used in	BART, T5, FLAN-T5, Pegasus	GPT-3/4, Gemini, Claude	Encoder-decoder generators are popular in machine translation, summarization, and tasks where structured synthesis is critical. Decoder-only generators dominate today’s open-source and API ecosystems (OpenAI, Google, Anthropic, Meta, Mistral, etc.)—they’re the backbone of most modern assistants, RAG pipelines, and chatbots.

Prompt Formatting for Generation

Up to now, we've focused on:

- How to **chunk** documents, so you have the right “building blocks” for retrieval.
- How to **retrieve** the most relevant, diverse, and up-to-date chunks.
- How to Generate by combining prompt and context.

This combination of prompt and context is not so straightforward and needs to be strategized.

So here's the last critical piece—**how you serve those chunks to the LLM**.

Even if you've perfectly retrieved the best possible context, if you hand it to the model as a messy, confusing prompt, the LLM may still hallucinate, miss key details, or output a generic answer.

Prompt formatting is the “plating” in the RAG kitchen—what and how you serve is as important as what you've prepared.

This is where we control the problem of hallucination.

1. Benefits of Prompt Engineering with RAG

1.1. Clear Structure Prevents Confusion

The LLM doesn't know which part of the input is “context” and which is “question” unless you tell it. A good prompt clearly separates:

- Retrieved chunks (context)
- The actual user query
- Any instructions (“Answer using only the provided context.”)

1.2. Reduces Hallucination and Repetition

If context and question are blended together, the LLM might ignore the details, repeat chunks verbatim, or make up answers.

Explicit prompt formatting helps the model ground its answer—using *your* retrieved facts.

1.3 Helps with Token Efficiency

You want to make every token count.

Formatting lets you prioritize the most important info and avoid wasting input space on boilerplate or poorly structured text.

Models like GPT-4 have strict token limits. Wasted tokens = more cost = weaker performance.

Well-structured prompts help us:

1. Prioritize top-ranked chunks
2. Avoid duplication or irrelevant metadata
3. Reduce boilerplate and verbose padding

1.4 Grounding

Grounding means making sure that a language model's answer is based on real, verifiable facts—not just its own “imagination” or patterns it learned during training.

In other words, a grounded response is:

- Traceable to specific context or data
- Factual, not fabricated
- As close as possible to what's actually in your documents, database, or provided knowledge

If the model “makes up” something that wasn't in the supplied context, it's not grounded.

2. Best Practices for RAG prompting

2.1 Segregate Roles

Principle: Make context, instructions, and questions clearly distinct with headers, lines, or even tables.

```
CONTEXT:
-----
Applicants must be at least 21 years old.
Minimum salary is ₹25,000/month.
Credit score required is above 700.
-----

USER QUESTION:
What is the minimum salary for a personal loan?

INSTRUCTIONS:
- Answer using only the provided context.
- Do not use external knowledge.
- Respond clearly and concisely.
```

ANSWER:

Notice how each part is visually separated.

2.2 Summarize Context Structure

Principle: Help the LLM “navigate” the facts by labeling sections, numbering, or summarizing sources.

```
REFERENCE SECTIONS:
=====
Section 1: Eligibility Criteria
Applicants must be at least 21 years old.

Section 2: Salary Requirement
Minimum salary is ₹25,000/month.

Section 3: Credit Score Requirement
Credit score required is above 700.
=====

USER QUESTION:
List all the requirements for a personal loan.

INSTRUCTIONS:
- Provide Reference section numbers in your answer.
```

LLM can “see” where to find each fact and can cite sections.

2.3 Set Explicit Instructions

Principle: Specify if the model should only use context, what to do if the answer is not found, and whether to cite sources.

```
INSTRUCTIONS:
- Use ONLY the context above.
- If the answer is not in the context, reply: "Not found in context."
- Cite the section number if relevant.
```

No ambiguity—the LLM is told what to do and what not to do.

2.4 Guide for Hallucination Risk

Principle: Use phrasing like “If not found, say: ‘Not found in context.’” to limit made-up answers.

```
If you cannot find the answer in the context, do not guess. Instead, write:  
"Not found in context."
```

2.5 Align with Task Complexity

Principle: Use simple context blocks for focused Q&A, and highly-structured or tabular formats for broad, compliance-heavy, or multi-fact queries.

Simple Context Example (for focused Q&A):

CONTEXT TABLE:

```
-----  
Year | Type   | Requirement  
2024 | Policy | Minimum salary is ₹25,000/month.  
2024 | Policy | Credit score required is above 700.  
2018 | Legacy | Minimum salary was ₹20,000/month.  
-----
```

USER QUESTION:

What is the current minimum salary for a personal loan?

INSTRUCTIONS:

- Use only the most recent 'policy' row to answer.
- Cite the year in your response.

3. Advanced Prompting with RAGs

3.1. Few-shot Examples (In-Context Learning)

What it is:

You include a few demonstration Q&A pairs in your prompt, showing the LLM *how* to answer questions using context.

Why it matters:

This teaches the model your desired style, depth, format, and helps prevent hallucination or off-topic answers—especially for tricky question types or when you want answers in a specific format.

Example:

```
few_shot_prompt = f"""
You are a lending policy expert. Answer each user question using ONLY the
context provided for each question.

Example 1
Context:
Applicants must be at least 21 years old.
Question: What is the minimum age to apply?
Answer: The minimum age to apply is 21 years old.

Example 2
Context:
Minimum salary is ₹25,000/month.
Question: What is the minimum salary for a personal loan?
Answer: The minimum salary is ₹25,000 per month.

---

Now, answer this question.

Context:
Applicants must have a credit score above 700.
Question: What credit score is required for a personal loan?
Answer:
"""
```

Here you make static few-shot examples with context because you can't do this in real time.

3.2 Stepwise Prompts (Chain-of-Thought or Multi-step Guidance)

What it is:

You break down a complex task into smaller, explicit steps within the prompt—helping the LLM reason step-by-step, instead of leaping to the final answer.

Why it matters:

For questions that require reasoning, synthesis, or multi-part answers (like eligibility checks, multi-factor decisions), stepwise prompts reduce mistakes and hallucinations.

Example:

```
stepwise_prompt = f"""
You are a compliance analyst. Use the context to check personal loan
```

```
eligibility.
```

```
Step 1: List all requirements stated in the context.
```

```
Step 2: For each requirement, say whether the user meets it (assume user is  
23 years old, salary ₹30,000/month, credit score 750).
```

```
Step 3: Summarize if the user is eligible or not.
```

```
Context:
```

```
Applicants must be at least 21 years old.
```

```
Minimum salary is ₹25,000/month.
```

```
Credit score required is above 700.
```

```
Answer:
```

```
"""
```

3.3 Multi-stage Prompting (LLM as an Orchestrated Workflow)

What it is:

You don't use the LLM just once, but in a sequence of stages. Each stage tackles a specific sub-task—such as query rewriting, summarizing context, or synthesizing information from multiple chunks/sources.

Why it matters:

This modularizes complex workflows, lets you check or “clean” LLM outputs between steps, and enables building more sophisticated pipelines.

Example Workflow:

Stage 1: Rephrase Query for Clarity

```
rephrase_prompt = f"""
```

```
You are an expert assistant. Rephrase the following user query to make it  
more specific and unambiguous.
```

```
Original Query:
```

```
"Can I get a top-up loan after three missed EMIs?"
```

```
Rephrased Query:
```

```
"""
```

```
# Use the output as the new query for retrieval
```

Stage 2: Summarize Retrieved Chunks

```
summarize_prompt = f"""
Summarize the key points from the following context for decision-making.

Context:
{retrieved_context}

Summary:
"""
# Use the summary in the final answer generation prompt
```

Stage 3: Synthesize Final Answer

```
final_prompt = f"""
Using ONLY the summarized context below, answer the user's question.

Summary:
{context_summary}

Question:
{user_query}

Answer:
"""
```

What not to do while Prompting with RAGs

What Not to Do	Problem/Effect	How to Fix It
Blending context and question without separators	Model treats everything as one sequence; may continue context rather than answer; unclear boundaries	Always separate context and question with clear headings, lines, or keywords (e.g., "CONTEXT:"/"QUESTION:")

Context overload (too many similar chunks)	LLM becomes vague, may ignore or summarize without details, can hallucinate from sheer volume	Select only the most relevant, diverse chunks; use chunk overlap wisely; apply MMR for diversity
Repeating facts in multiple chunks	LLM may repeat the same information in answers; output feels redundant or circular	Remove duplicate or near-duplicate chunks before prompting; use deduplication or re-ranking
Unordered context	Model might answer from an irrelevant section or get “distracted”; factual accuracy drops	Order chunks by recency, importance, or logical flow; provide section labels or metadata
No fallback for missing info	Model hallucinates or invents answers when fact is missing	Add explicit instructions: “If not found, reply: ‘Not found in context.’”
Mixing formats/styles within prompt	Inconsistent output, possible confusion for LLM	Use consistent prompt structure and language for context, question, and instructions
No explicit grounding instructions	LLM might mix knowledge from training with provided context, reducing trust and accuracy	Directly instruct: “Use ONLY the above context. Do not use prior knowledge.”

Key Takeaway:

Each LLM call has a focused role, improving control, explainability, and robustness in your RAG pipeline.

Technique	When to Use	Benefits
Few-shot	Control output style, train LLM “by example”	Reduces format drift, hallucination
Stepwise	Tasks needing logic, multi-step answers	Transparent, less error-prone

Multi-stage	Complex, modular workflows	Modular, easy to debug/tune
-------------	----------------------------	-----------------------------

Session Summary: RAG Pipeline – Document & Retrieval Strategies

- **RAG architecture** relies on two stages: retrieving relevant context (retriever) and generating answers (generator), with the generator being either encoder-decoder or decoder-only.
- **Chunking** is critical—documents must be split into meaningful pieces (fixed-size, hierarchical, or semantic) so retrieval and LLM grounding work well.
- **Poor chunking** (chunks that are too long, too short, or cut mid-meaning) causes lost information, retrieval failures, and LLM hallucinations.
- **Similarity metrics** (like cosine similarity) determine what is considered “relevant”—the right metric avoids “almost right” answers and guides high-quality retrieval.
- **Thresholding** filters out weak matches, preventing noise from reaching the LLM.
- **Maximum Marginal Relevance (MMR)** ensures the set of retrieved chunks is both relevant and diverse, not just repetitive.
- **Re-ranking and metadata filtering** let you prioritize the most current, trustworthy, and appropriate information, further improving answer quality.
- **Advanced frameworks** like LangChain and LlamaIndex operationalize these strategies, making robust RAG systems easier to build.
- **The core lesson:** Retrieval quality is determined just as much by how you prepare and select your chunks as by the model you use to generate answers.